

✓ Portfolio Construction and Hedging Using Duration and Convexity

This project involves managing a \$100 million portfolio comprising ten U.S. Treasury bonds of different maturities. The goal is to construct and hedge the portfolio using duration and convexity.

The following outlines the steps taken, including bond selection, calculations, portfolio construction, hedging strategies, and Monte Carlo simulations using the Vasicek interest rate model.

```
import numpy as np
import pandas as pd
import datetime as dt
from scipy.optimize import minimize
import matplotlib.pyplot as plt
```

✓ I.a. Rationale for Choice of Bonds and Futures Contracts

Ten U.S. Treasury Bonds

In constructing our \$100 million bond portfolio, we carefully selected ten U.S. Treasury bonds with varying maturities, coupon rates, and yields to maturity. The primary considerations in our selection process were:

1. Maturity Diversification:

We chose bonds with maturities ranging from less than one year to over 15 years. This diversification across the yield curve helps spread interest rate risk and allows the portfolio to benefit from different segments of the term structure. Short-term bonds provide liquidity and reduce duration, while long-term bonds increase duration and the potential for capital gains if rates fall.

2. Coupon Rates:

The bonds include both zero-coupon bonds and bonds with coupons ranging from 3.375% to 4.625%. Including zero-coupon bonds allows us to have instruments with higher sensitivity to interest rates, which is useful in managing the portfolio's overall duration. Bonds with higher coupons provide steady income and can help reduce duration.

3. Liquidity:

All selected bonds are U.S. Treasury securities, which are among the most liquid fixed-income instruments in the market. High liquidity ensures that the bonds can be bought and sold with minimal transaction costs and price impact.

4. Alignment with Investment Objectives:

Our goal is to construct a portfolio with a target modified duration of five years. By selecting a mix of bonds with different durations, we can adjust the portfolio's overall duration to meet this target. The diversity in maturities and coupons allows us to fine-tune the portfolio's duration and convexity.

Selection of Futures Contracts

To hedge the interest rate risk of our bond portfolio, we selected two futures contracts:

1. 3-Month SOFR Futures
2. 10-Year U.S. Treasury Note Futures (T-Note Futures)

For 3-Month SOFR Futures

- Alignment with Short-Term Rates:

The SOFR futures are based on the Secured Overnight Financing Rate, which reflects the cost of borrowing cash overnight collateralized by Treasury securities. It is a benchmark for short-term interest rates.

- Hedging Short-Term Interest Rate Exposure:

The shorter-maturity bonds in our portfolio (e.g., zero-coupon bonds maturing within one to two years) are sensitive to changes in short-term rates. The 3-month SOFR futures effectively hedges this exposure.

- Liquidity and Market Acceptance:

SOFR futures have gained significant liquidity since their introduction, making them suitable for hedging purposes. They are widely accepted in the market as a replacement for Eurodollar futures.

- Maturity Alignment:

The maturity dates of the SOFR futures align with our hedging horizon. Since we are considering a hedging period that includes short-term rate movements, the 3-month SOFR futures are appropriate.

For 10-Year U.S. Treasury Note Futures

- Hedging Long-Term Interest Rate Exposure:

The longer-maturity bonds in our portfolio (e.g., bonds maturing in 2029 to 2040) are more sensitive to changes in long-term rates. The 10-year T-Note futures effectively hedge this exposure.

- High Liquidity:

The 10-year T-Note futures are among the most liquid futures contracts in the world, ensuring efficient execution and minimal transaction costs.

- Effective Duration Matching:

The duration of the T-Note futures aligns well with the duration of the long-term bonds in our portfolio, making them effective instruments for hedging interest rate risk.

- Availability of CTD Bond:

The existence of a well-defined Cheapest-to-Deliver (CTD) bond for the T-Note futures allows for precise calculation of the futures' duration and convexity, enhancing the accuracy of our hedging strategy.

✓ I.b. Bond Data

✓ I.b.i. Price conversion from ticks to dollars

Let's use our 2-Year U.S. Treasury Note for each example, represented by Bond5, it's quoted price is given as: 100-10 1/8

Conversion Steps

1. Separate the Whole Number and Fraction:

- Whole Number: 100
- Fraction: 10 1/8

2. Convert the Fraction to Decimal:

- The fractions are in 32nds. So 10 represents $\frac{10}{32}$.
- The $\frac{1}{8}$ indicates an additional eighth of a 32nd, which is $\frac{1}{8} \times \frac{1}{32} = \frac{0.125}{32}$.

Therefore, the fractional part in decimal is:

$$\text{Fraction in Decimal} = \frac{10}{32} + \frac{0.125}{32}$$

3. Compute the Decimal Equivalent:

- Calculate the decimal value of the fractional part:

$$\frac{10}{32} = 0.3125$$
$$\frac{0.125}{32} = 0.00390625$$

$$\text{Total Fraction in Decimal} = 0.3125 + 0.00390625 = 0.31640625$$

4. Add the Whole Number Part:

- Combine the whole number and the fractional decimal:

$$\text{Decimal Price} = 100 + 0.31640625 = 100.31640625$$

The quoted bond price of 100-10 1/8 converts to a decimal price of \$100.31640625.

We begin by compiling data for ten U.S. Treasury bonds with varying maturities:

```
# Bond Data
```

```
bond_data = [  
    {'BondID': 'Bond1', 'CouponRate': 0.000, 'MaturityDate': '11/26/2024', 'YTM': 4.731545, 'Price': 99.2665208},  
    {'BondID': 'Bond2', 'CouponRate': 0.000, 'MaturityDate': '12/26/2024', 'YTM': 4.611012, 'Price': 98.9158229},  
    {'BondID': 'Bond3', 'CouponRate': 0.000, 'MaturityDate': '03/27/2025', 'YTM': 4.38962, 'Price': 97.9041736},  
    {'BondID': 'Bond4', 'CouponRate': 0.000, 'MaturityDate': '09/04/2025', 'YTM': 3.928786, 'Price': 96.4487396},  
    {'BondID': 'Bond5', 'CouponRate': 3.75, 'MaturityDate': '08/31/2026', 'YTM': 3.576137, 'Price': 100.316406},  
    {'BondID': 'Bond6', 'CouponRate': 3.375, 'MaturityDate': '09/15/2027', 'YTM': 3.479505, 'Price': 99.707031},  
    {'BondID': 'Bond7', 'CouponRate': 4.25, 'MaturityDate': '02/28/2029', 'YTM': 3.495839, 'Price': 103.058594},  
    {'BondID': 'Bond8', 'CouponRate': 3.75, 'MaturityDate': '08/31/2031', 'YTM': 3.609183, 'Price': 100.8359375},  
    {'BondID': 'Bond9', 'CouponRate': 3.875, 'MaturityDate': '08/15/2034', 'YTM': 3.752213, 'Price': 101.0},  
    {'BondID': 'Bond10', 'CouponRate': 4.625, 'MaturityDate': '02/15/2040', 'YTM': 3.940557, 'Price': 107.828125}
```




✓ I.b.ii. Calculating Semi-Annual Coupon Payments

Bond Details:

- Bond ID: Bond5
- Coupon Rate: 3.75%
- Maturity Date: August 31, 2026
- Face Value: \$1,000
- Settlement Date: October 3, 2024
- Coupon Payment Frequency: Semi-Annual

The semi-annual coupon payment for a bond is calculated using the formula:

$$\text{Coupon Payment} = \frac{\text{Face Value} \times \text{Annual Coupon Rate}}{2}$$

Substituting the values:

$$\text{Coupon Payment} = \frac{1000 \times 0.0375}{2} = 18.75$$

Using a function to automate it, we get the same thing :

```
# Calculating Semi-Annual Coupon Payments
def semi_annual_coupon(face_value, annual_coupon_rate):
    return (face_value * annual_coupon_rate) / 2

# For Bond5
face_value = 1000 # Assuming a face value of $1,000
annual_coupon_rate = bond_df.loc[bond_df['BondID'] == 'Bond5', 'CouponRate'].values[0]
coupon_payment = semi_annual_coupon(face_value, annual_coupon_rate)
print(f"Semi-Annual Coupon Payment for Bond5: ${coupon_payment:.2f}")
```

🔗 Semi-Annual Coupon Payment for Bond5: \$18.75

✓ I.b.iii. Timeline of Future Cash Flows

For each bond, we generate future cash flows:

```
# Functions to generate cash flows for all bonds
def generate_cash_flows(bond, face_value=1000, settlement_date=pd.to_datetime('09/30/2024')):
    maturity_date = bond['MaturityDate']
    annual_coupon_rate = bond['CouponRate']
    coupon_payment = semi_annual_coupon(face_value, annual_coupon_rate)

    if annual_coupon_rate == 0:
        # Zero-coupon bond
        if maturity_date <= settlement_date:
```

```

        return pd.DataFrame(columns=['Date', 'CashFlow'])
    cash_flows = pd.DataFrame({'Date': [maturity_date], 'CashFlow': [face_value]})
else:
    # Generate coupon dates from maturity date backwards
    dates = []
    date = maturity_date
    delta = pd.DateOffset(months=6) # Semi-annual coupons
    while date > settlement_date:
        dates.append(date)
        date -= delta
    dates = sorted(dates)
    cash_flows = pd.DataFrame({'Date': dates})
    cash_flows['CashFlow'] = coupon_payment
    # Adjust last cash flow to include face value
    cash_flows.loc[cash_flows.index[-1], 'CashFlow'] += face_value

# Add 'Time' column
cash_flows['Time'] = (cash_flows['Date'] - settlement_date).dt.days / 365.25

return cash_flows

cash_flow_dict = {}

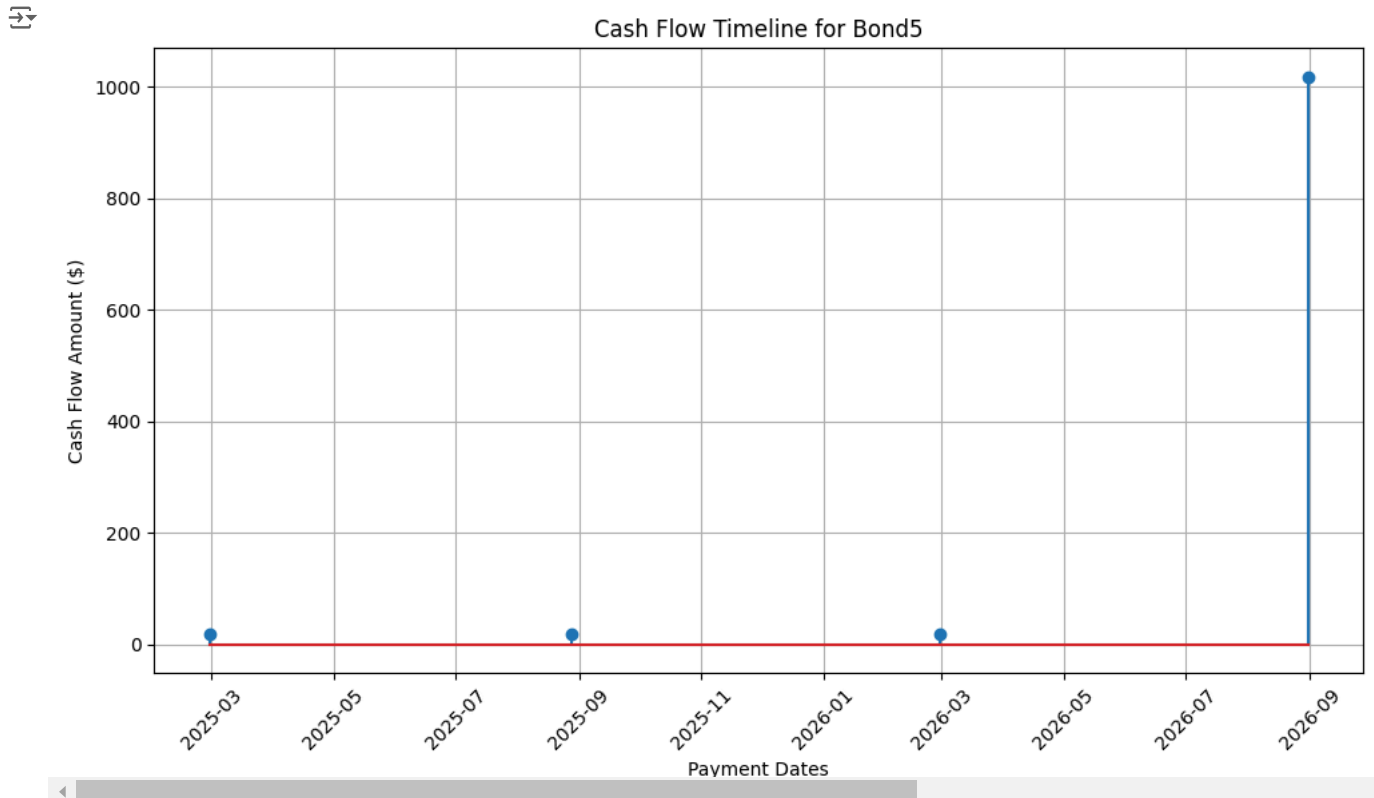
face_value = 1000
settlement_date = pd.to_datetime('09/30/2024')

# Loop to generate cash flows
for index, bond in bond_df.iterrows():
    cash_flows = generate_cash_flows(bond, face_value, settlement_date)
    cash_flow_dict[bond['BondID']] = cash_flows

# Retrieve the cash flows for Bond5
cash_flows_bond5 = cash_flow_dict['Bond5']

# Plotting Cash Flow Timeline for Bond5
plt.figure(figsize=(10, 6))
plt.stem(cash_flows_bond5['Date'], cash_flows_bond5['CashFlow'])
plt.xlabel('Payment Dates')
plt.ylabel('Cash Flow Amount ($)')
plt.title('Cash Flow Timeline for Bond5')
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.show()

```



The bond pays coupons every six months until maturity. The future cash flows and their payment dates are as follows:

Feb 28, 2025 Aug 31, 2025 Feb 28, 2026 Aug 31, 2026

18.75|18.75 18.75|1,018.75

Each date represents when the cash flows occur, and the amounts are the expected payments on those dates.

✓ I.b.iv. Accrued Interest and Invoice Price

The accrued interest is calculated as:

$$\text{Accrued Interest} = \text{Coupon Payment} \times \frac{\text{Days Since Last Coupon}}{\text{Days Between Coupons}}$$

For **Bond5**, assuming:

- Last Coupon Date: August 31, 2024
- Next Coupon Date: February 28, 2025
- Settlement Date: September 30, 2024

```
# Accrued Interest and Invoice Price
def accrued_interest(coupon_payment, last_coupon_date, settlement_date, next_coupon_date):
    days_between_coupons = (next_coupon_date - last_coupon_date).days
    days_since_last_coupon = (settlement_date - last_coupon_date).days
    return coupon_payment * (days_since_last_coupon / days_between_coupons)

# For Bond5
last_coupon_date = pd.to_datetime('08/31/2024')
next_coupon_date = pd.to_datetime('02/28/2025')
settlement_date = pd.to_datetime('09/30/2024')
accrued = accrued_interest(coupon_payment, last_coupon_date, settlement_date, next_coupon_date)

# Invoice Price
clean_price = bond_df.loc[bond_df['BondID'] == 'Bond5', 'Price'].values[0]
invoice_price = clean_price + accrued / 10 # Adjusting for price per $100 face value
print(f"Accrued Interest for Bond5: ${accrued:.2f}")
print(f"Invoice Price for Bond5: ${invoice_price:.2f} per $100 face value")
```

→ Accrued Interest for Bond5: \$3.11
Invoice Price for Bond5: \$100.63 per \$100 face value

✓ I.c. Duration and Convexity Calculations

Duration measures the sensitivity of a bond's price to changes in interest rates, while **convexity** measures the rate of change of duration with respect to interest rates.

I.c.i. Duration and Convexity Formulas

1. Discount Factors and Present Values:

- We calculate the discount factors using the formula $(1 + \frac{r}{\text{frequency}})^{-t}$, where t is the period number.
- The present value of each cash flow is then calculated, and the total present value P_0 is obtained.

2. Weights w_t :

- We compute the weights $w_t = \frac{\text{PV of Cash Flow at } t}{P_0}$ for each period.

3. Duration Calculation:

$$\text{Duration} = \sum_{t=1}^T t \times w_t$$

- Since we have semi-annual periods, we divide by the frequency to convert the duration back to years.

4. Convexity Calculation:

$$C = \frac{1}{(1 + r/\text{frequency})^2} \times \sum_{t=1}^T t \times (t + 1) \times w_t$$

- We adjust for the compounding frequency in both the numerator and denominator.

5. Zero-Coupon Bond Convexity:

- For zero-coupon bonds, the convexity simplifies to:

$$C_{\text{zero-coupon}} = \frac{T \times (T + 1)}{(1 + r/\text{frequency})^2}$$

I.c.ii. Calculating Durations and Convexities for All Bonds

We extend the calculations to all bonds in the portfolio:

```
# Duration and Convexity Calculations
def calculate_duration_convexity(cash_flows, yield_to_maturity, settlement_date):
    frequency = 2 # Semi-annual compounding
    r = yield_to_maturity
    cash_flows = cash_flows.copy()

    # Calculate time in years from settlement date to each cash flow
    cash_flows['Time'] = (cash_flows['Date'] - settlement_date).dt.days / 365.25
    # Periods = Time in years * frequency
    cash_flows['Period'] = cash_flows['Time'] * frequency # May not be integer

    # Discount factors
    cash_flows['DiscountFactor'] = (1 + r / frequency) ** (-cash_flows['Period'])
    cash_flows['PV'] = cash_flows['CashFlow'] * cash_flows['DiscountFactor']
    P_0 = cash_flows['PV'].sum()

    # Weights
    cash_flows['w_t'] = cash_flows['PV'] / P_0

    # Duration Calculation
    cash_flows['t_wt'] = cash_flows['Period'] * cash_flows['w_t']
    duration = cash_flows['t_wt'].sum() / frequency # Adjust for frequency

    modified_duration = duration / (1+r/frequency)

    # Convexity Calculation
    cash_flows['t_t1_wt'] = cash_flows['Period'] * (cash_flows['Period'] + 1) * cash_flows['w_t']
    convexity = cash_flows['t_t1_wt'].sum() / ((1 + r / frequency) ** 2 * frequency ** 2)

    return duration, convexity, modified_duration

# Calculate durations and convexities for all bonds
durations = []
convexities = []

# Calculate for all bonds
for index, bond in bond_df.iterrows():
    bond_id = bond['BondID']
    cash_flows = cash_flow_dict[bond_id]

    if cash_flows.empty:
        bond_df.at[index, 'Duration'] = 0
        bond_df.at[index, 'Convexity'] = 0
        bond_df.at[index, 'Modified_Duration'] = 0
        continue

    duration, convexity, modified_dur = calculate_duration_convexity(cash_flows, bond['YTM'], settlement_date)
    bond_df.at[index, 'Duration'] = duration
    bond_df.at[index, 'Convexity'] = convexity
    bond_df.at[index, 'Modified_Duration'] = modified_dur

# Display the Bond5 Duration and Convexity
bond5_data = bond_df[bond_df['BondID'] == 'Bond5'][['BondID', 'Duration', 'Modified_Duration', 'Convexity']]
print(bond5_data)
```

```
➡ BondID Duration Modified_Duration Convexity
4 Bond5 1.861792 1.829086 4.303202
```

✓ I.c.iii. Comparison with Bloomberg's Calculations

In Bloomberg, we have a Modified Duration of **1.83** and a Convexity of **0.043**.

Bloomberg typically reports Macaulay or modified duration on an annual basis, while we calculate duration based on semi-annual coupon payments. As a result, we observed a lower duration with semi-annual coupons. This is because the investor receives cash flows more frequently, which reduces the time-weighted impact of future payments and diminishes the bond's sensitivity to interest rate changes, ultimately **lowering the duration**.

Convexity, like duration, will also be affected by the frequency of coupon payments (annual vs. semi-annual). Generally, a bond with semi-annual coupons will have **slightly higher convexity** than a bond with annual coupons. This is because convexity measures the curvature of the relationship between the bond's price and interest rates. When cash flows are more frequent, the present value of the payments is more sensitive to changes in interest rates, thereby increasing convexity.

I.d. Duration and Convexity for Futures

I.d.i. Calculating Futures Prices

```
def convert_treasury_future_price(price_str):
    if '+' in price_str:
        price_str = price_str.replace('+', '.5')
    whole, fraction = price_str.split('-')
    whole = int(whole)
    fraction = float(fraction)
    decimal_price = whole + fraction / 32
    return decimal_price

# For 10-Year U.S. Treasury Note Futures
futures_price_str = '111-23+'
futures_decimal_price = convert_treasury_future_price(futures_price_str)
print(f"Decimal Futures Price: {futures_decimal_price}")

# Futures contract value
futures_price_tnote = futures_decimal_price * 1000 # Each point is $1,000
print(f"Futures Price for T-Note: ${futures_price_tnote:.2f}")
```

Decimal Futures Price: 111.734375
Futures Price for T-Note: \$111734.38

✓ I.d.ii Calculating Bond Duration and Convexity for the CTD Bond

Identify the Cheapest-to-Deliver (CTD) Bond

The Cheapest-to-Deliver (CTD) bond is the bond that minimizes the cost to the seller (short position) when delivering into a futures contract. Using bloomberg, we found the CTD of our interest rate future and it's following information :

- **Coupon Rate:** 6%
- **Maturity Date:** November 15, 2031
- **YTM:** 3.5%

We first need to calculate the duration and convexity for the CTD, then calculate the conversion factor in order to obtain the duration and convexity of the interest rate future

```
def calculate_CTD_duration_convexity(settlement_date, maturity_date, coupon_rate, face_value, yield_to_maturity, frequency=2):
    # Generate coupon payment dates
    coupon_dates = pd.date_range(start=settlement_date + pd.DateOffset(days=1),
                                  end=maturity_date, freq=f'{12 // frequency}ME')
    if coupon_dates.empty or coupon_dates[-1] != maturity_date:
        coupon_dates = coupon_dates.append(pd.to_datetime([maturity_date]))

    # Calculate cash flows
    coupon_payment = face_value * coupon_rate / frequency
    cash_flows = pd.DataFrame({'Date': coupon_dates})
    cash_flows['CashFlow'] = coupon_payment
    cash_flows.loc[cash_flows['Date'] == maturity_date, 'CashFlow'] += face_value

    # Call calculate_duration_convexity to get duration and convexity
    duration, convexity, _ = calculate_duration_convexity(cash_flows, yield_to_maturity, settlement_date)

    # Total present value (PV)
    # Periods = Time in years * frequency
    cash_flows['Time'] = (cash_flows['Date'] - settlement_date).dt.days / 365.25
    cash_flows['Period'] = cash_flows['Time'] * frequency # May not be integer
    # Discount factors
    cash_flows['DiscountFactor'] = (1 + yield_to_maturity / frequency) ** (-cash_flows['Period'])
    cash_flows['PV'] = cash_flows['CashFlow'] * cash_flows['DiscountFactor']
    pv_total = cash_flows['PV'].sum()

    return duration, convexity, pv_total

settlement_date = pd.to_datetime('09/30/2024')
maturity_date_ctd = pd.to_datetime('11/15/2031')
coupon_rate_ctd = 0.06 # 6%
face_value_ctd = 100
ytm_ctd = 0.035 # 3.5%

duration_ctd, convexity_ctd, pv_ctd = calculate_CTD_duration_convexity(
    settlement_date, maturity_date_ctd, coupon_rate_ctd, face_value_ctd, ytm_ctd)

print(f"CTD Bond Duration: {duration_ctd:.4f} years")
print(f"CTD Bond Convexity: {convexity_ctd:.4f}")
```


CTD Bond Duration: 5.8969 years
CTD Bond Convexity: 40.8588

✓ Calculating the Conversion Factor

The conversion factor is used to adjust the duration and convexity of the futures contract to reflect those of the CTD bond :

$$CF = \frac{\text{Price of the Bond at 6\% Yield}}{\text{Face Value of the Bond}}$$

```
def calculate_conversion_factor(coupon_rate, maturity_date, settlement_date):  
    # Price of the bond if the yield were 6%  
    _, price_at_6_percent = calculate_CTD_duration_convexity(  
        settlement_date, maturity_date, coupon_rate, face_value_ctd, 0.06)  
    conversion_factor = price_at_6_percent / face_value_ctd  
    return conversion_factor  
  
conversion_factor_ctd = calculate_conversion_factor(coupon_rate_ctd, maturity_date_ctd, settlement_date)  
print(f"Conversion Factor for CTD Bond: {conversion_factor_ctd:.6f}")
```

Conversion Factor for CTD Bond: 1.043079

✓ I.d.iii. Calculating Duration and Convexity of the Futures Contracts

The duration and convexity of the futures contract are adjusted based on the CTD bond and conversion factor:

- **Duration of Futures Contract:**

$$\text{Duration}_{\text{CTD}} \times \left(\frac{\text{Price}_{\text{CTD}}}{\text{Futures Price} \times \text{Conversion Factor}} \right)$$

- **Convexity of Futures Contract:** Similar adjustment applies.

However, since the futures contract is marked to market daily, and delivery happens at expiration, the futures duration is approximately equal to the duration of the CTD bond divided by the conversion factor.

```
# Futures Price  
futures_price_tnote = futures_decimal_price * 1000 # Price per contract  
# Adjusted Duration  
duration_futures_tnote = duration_ctd / conversion_factor_ctd  
convexity_futures_tnote = convexity_ctd / conversion_factor_ctd  
  
print(f"Duration of T-Note Futures Contract: {duration_futures_tnote:.4f} years")  
print(f"Convexity of T-Note Futures Contract: {convexity_futures_tnote:.4f}")
```

Duration of T-Note Futures Contract: 5.6534 years
Convexity of T-Note Futures Contract: 39.1714

✓ I.d.iv. Calculating Duration and Convexity of the 3-Month SOFR Futures

SOFR futures are based on the Secured Overnight Financing Rate and are similar to Eurodollar futures but based on SOFR instead of LIBOR. The price of a SOFR futures contract is quoted as:

$$\text{Price} = 100 - \text{Annualized SOFR}$$

The underlying asset is an average of daily SOFR rates over a 3-month period.

Calculating Duration

The duration of a 3-month SOFR futures contract is approximately equal to the time to maturity of the contract since the contract is settled in cash based on the average SOFR over the contract period.

$$\text{Duration} \approx \frac{\text{Time to Maturity}}{1 + r}$$

Calculating Convexity

The convexity of the SOFR futures contract is negligible due to its short maturity and linear payoff structure. However, for completeness, we'll calculate it.

For short-term instruments, the convexity is approximately:

$$\text{Convexity} \approx \frac{(\text{Time to Maturity})^2}{(1 + r)^2}$$

```
# Time to maturity for SOFR futures  
settlement_date = pd.to_datetime('09/30/2024')
```

```
futures_maturity_sofr = pd.to_datetime('03/18/2025')
time_to_maturity_sofr = (futures_maturity_sofr - settlement_date).days / 360

price_SOFR = 96.05 #using our 3 month SOFR Futures
r_sofr = 100 - price_SOFR

duration_futures_sofr = time_to_maturity_sofr / (1 + r_sofr)
convexity_futures_sofr = (time_to_maturity_sofr ** 2) / ((1 + r_sofr) ** 2)

print(f"Duration of SOFR Futures Contract: {duration_futures_sofr:.6f} years")
print(f"Convexity of SOFR Futures Contract: {convexity_futures_sofr:.8f}")
```

```
➡ Duration of SOFR Futures Contract: 0.094837 years
Convexity of SOFR Futures Contract: 0.00899411
```

✓ I.e. Duration and Convexity for all Bonds

```
# Display the all Bond Durations and Convexities
print(bond_df[['BondID', 'Duration', 'Modified_Duration', 'Convexity']])
```

```
➡
```

	BondID	Duration	Modified_Duration	Convexity
0	Bond1	0.156057	0.152451	0.097705
1	Bond2	0.238193	0.232825	0.167997
2	Bond3	0.487337	0.476871	0.460721
3	Bond4	0.928131	0.910251	1.274913
4	Bond5	1.861792	1.829086	4.303202
5	Bond6	2.834801	2.786326	9.336429
6	Bond7	4.062622	3.992831	18.757791
7	Bond8	6.139932	6.031095	42.192667
8	Bond9	8.267995	8.115735	77.708563
9	Bond10	11.289189	11.071058	153.789201

✓ II. Portfolio Construction

II.a. Criteria for Portfolio Construction

Our objective in constructing the bond portfolio was to achieve a target modified duration of five years while investing a total of \$100 million. The criteria and considerations in determining the positions in each bond were as follows:

Factors Considered

1. Duration Matching:

The primary goal was to achieve a portfolio modified duration of five years. We calculated the duration of each bond and used these values to optimize the portfolio's overall duration.

2. Diversification Across Maturities:

We included bonds with varying maturities, from less than one year to over 15 years. This diversification helps spread interest rate risk and reduces the portfolio's sensitivity to changes in any particular segment of the yield curve.

3. Coupon Rates:

Bonds with different coupon rates were selected to balance income generation and price sensitivity. Lower-coupon bonds and zero-coupon bonds have higher durations, contributing more to the portfolio's duration target.

4. Liquidity Considerations:

All selected bonds are U.S. Treasury securities, ensuring high liquidity. This allows for efficient rebalancing and reduces transaction costs.

5. Risk Management:

By diversifying across different bonds, we mitigated the idiosyncratic risk associated with any single bond. The allocation aimed to avoid over-concentration in any single maturity or coupon rate.

II.b. Optimization Problem

Objective

Our goal is to construct a bond portfolio that achieves a modified duration of 5 years. Given a set of bonds with known durations, we need to determine the optimal allocation (weights) to each bond so that the overall portfolio duration matches the target duration as closely as possible.

To accomplish this, we formulate an optimization problem where we minimize the squared difference between the portfolio's duration and the target duration. The objective function is:

$$\min_{\mathbf{w}} \left(\sum_{i=1}^n w_i D_i - D_T \right)^2$$

Where:

- w_i = weight of bond i in the portfolio
- D_i = duration of bond i
- D_T = target duration (5 years)
- n = total number of bonds

Constraints

The optimization is subject to the following constraints:

1. Full Investment Constraint:

$$\sum_{i=1}^n w_i = 1$$

This ensures that the total allocation across all bonds sums up to 100% of the portfolio, meaning all available capital is fully invested.

2. Non-Negative Weights Constraint:

$$w_i \geq 0 \quad \text{for all } i = 1, 2, \dots, n$$

This prohibits short-selling, ensuring that all positions are long.

Rationale Behind the Objective Function

The objective function represents the squared deviation of the portfolio's duration from the target duration. By minimizing this, we aim to find the set of weights that brings the portfolio duration as close as possible to 5 years.

- **Squared Difference:** We use the squared difference to ensure that deviations in both directions (above or below the target duration) are penalized equally and to make the objective function differentiable, which is beneficial for optimization algorithms.
- **Convexity:** The objective function is convex because it is a quadratic function of $\mathbf{w}$. This property guarantees that any local minimum found is also the global minimum, simplifying the optimization process.

Solution Approach

We use numerical optimization techniques to solve this problem. Specifically, we employ the minimize function from the scipy.optimize library in Python, which allows us to define custom objective functions, constraints, and bounds.

```
# Construct the portfolio
prices = bond_df['Price'].values
durations = bond_df['Duration'].values

# Objective function
def objective(weights):
    portfolio_duration = np.sum(weights * durations)
    return (portfolio_duration - 5) ** 2

# Constraints: sum of weights = 1, weights >= 0
constraints = {'type': 'eq', 'fun': lambda x: np.sum(x) - 1}
bounds = [(0, 1) for _ in range(len(prices))]
x0 = np.full(len(prices), 1 / len(prices))

# Optimization
result = minimize(objective, x0, bounds=bounds, constraints=constraints)
weights = result.x

# Add weights to bond_df
bond_df['PortfolioWeight'] = weights
bond_df['CashFlow'] = bond_df['BondID'].map(cash_flow_dict)

# Allocate $100 million
portfolio_value = 100_000_000
bond_df['DollarInvestment'] = bond_df['PortfolioWeight'] * portfolio_value

# Portfolio Duration and Convexity
portfolio_duration = np.sum(weights * durations)
portfolio_convexity = np.sum(weights * bond_df['Convexity'].values)
print(f"Portfolio Duration: {portfolio_duration:.4f} years")
print(f"Portfolio Convexity: {portfolio_convexity:.4f}")

# Print the final weights
print("Weights allocated to each bond:")
for i, weight in enumerate(weights):
    print(f"Bond {i+1}: {weight:.4f}")
```

```
➦ Portfolio Duration: 5.0000 years
Portfolio Convexity: 52.8512
```

Weights allocated to each bond:
Bond 1: 0.0821
Bond 2: 0.0821
Bond 3: 0.0821
Bond 4: 0.0821
Bond 5: 0.0821
Bond 6: 0.0821
Bond 7: 0.0821
Bond 8: 0.0821
Bond 9: 0.0821
Bond 10: 0.2613

Allocation Results

- **Higher Weight to Longer-Duration Bonds:** Bonds with higher durations were assigned higher weights to increase the overall portfolio duration.
- **Balancing with Shorter-Duration Bonds:** Bonds with shorter durations were included to provide liquidity and reduce volatility. This balance ensures that the portfolio is not overly sensitive to long-term interest rate movements.
- **Avoiding Over-Concentration:** Constraints were applied to prevent allocating an excessively high percentage of the portfolio to any single bond, promoting diversification.

✓ III. Hedging Duration with Futures Contracts

To hedge the interest rate risk of our bond portfolio, we selected two futures contracts:

1. 3-Month SOFR Futures
2. 10-Year U.S. Treasury Note Futures (T-Note Futures)

III.1. Hedging Duration using 3-Month SOFR Futures

III.1.a. Selection of 3-Month SOFR Futures

- **Alignment with Short-Term Rates:**

The SOFR futures are based on the Secured Overnight Financing Rate, which reflects the cost of borrowing cash overnight collateralized by Treasury securities. It is a benchmark for short-term interest rates.

- **Hedging Short-Term Interest Rate Exposure:**

The shorter-maturity bonds in our portfolio (e.g., zero-coupon bonds maturing within one to two years) are sensitive to changes in short-term rates. The 3-month SOFR futures effectively hedges this exposure.

- **Liquidity and Market Acceptance:**

SOFR futures have gained significant liquidity since their introduction, making them suitable for hedging purposes. They are widely accepted in the market as a replacement for Eurodollar futures.

- **Maturity Alignment:**

The maturity dates of the SOFR futures align with our hedging horizon. Since we are considering a hedging period that includes short-term rate movements, the 3-month SOFR futures are appropriate.

III.1.b. Hedging Duration using 3-Month SOFR Futures

We aim to adjust the portfolio duration from 5 years to 1 year.

Calculations:

$$\text{Number of Futures Contracts} = - \frac{\Delta \text{Dollar Duration}}{\text{Dollar Duration per Futures Contract}}$$

```
# Hedging Parameters
portfolio_value = 100_000_000
initial_portfolio_duration = 5 # Current duration of 5 years
target_duration_sofr = 1 # Target portfolio duration after SOFR hedge

# Dollar Duration of the Portfolio before SOFR Hedge
dollar_duration_portfolio = (portfolio_value * initial_portfolio_duration) / 100

# Dollar Duration Target (1-year duration)
dollar_duration_target_sofr = (portfolio_value * target_duration_sofr) / 100

# Change in Dollar Duration Needed to Hedge using SOFR
dollar_duration_needed_sofr = dollar_duration_target_sofr - dollar_duration_portfolio

# SOFR Futures Details
contract_size_sofr = 25 # $25 per basis point
```

```
dollar_duration_futures_sofr = contract_size_sofr * duration_futures_sofr * 100 # Adjusted for 100 basis points
```

```
# Number of SOFR Futures Contracts Needed  
num_contracts_sofr = -dollar_duration_needed_sofr / dollar_duration_futures_sofr  
num_contracts_sofr = int(round(num_contracts_sofr))  
print(f"Number of 3-Month SOFR Futures Contracts Needed: {num_contracts_sofr}")
```

➦ Number of 3-Month SOFR Futures Contracts Needed: 16871

✓ III.2. Hedging Duration using Interest Rate Futures

III.1.a. Selection of Interest Rate Futures

- Alignment with Short-Term Rates:

The SOFR futures are based on the Secured Overnight Financing Rate, which reflects the cost of borrowing cash overnight collateralized by Treasury securities. It is a benchmark for short-term interest rates.

- Hedging Short-Term Interest Rate Exposure:

The shorter-maturity bonds in our portfolio (e.g., zero-coupon bonds maturing within one to two years) are sensitive to changes in short-term rates. The 3-month SOFR futures effectively hedges this exposure.

- Liquidity and Market Acceptance:

SOFR futures have gained significant liquidity since their introduction, making them suitable for hedging purposes. They are widely accepted in the market as a replacement for Eurodollar futures.

- Maturity Alignment:

The maturity dates of the SOFR futures align with our hedging horizon. Since we are considering a hedging period that includes short-term rate movements, the 3-month SOFR futures are appropriate.

III.1.b. Identification of Cheapest to Deliver Bond

As said before, the Cheapest-to-Deliver (CTD) bond is the bond that minimizes the cost to the seller (short position) when delivering into a futures contract. Using bloomberg, we found the CTD of our interest rate future and it's following information :

- **Coupon Rate:** 6%
- **Maturity Date:** November 15, 2031
- **YTM:** 3.5%

III.1.c. Hedging Duration using Interest rate Futures

We aim to adjust the portfolio duration from 5 years to 1 year.

Calculations:

$$\text{Number of Futures Contracts} = - \frac{\Delta \text{Dollar Duration}}{\text{Dollar Duration per Futures Contract}}$$

```
# Hedging Parameters  
# Assuming the portfolio is now at 1-year duration after SOFR futures hedge  
portfolio_value = 100_000_000  
target_duration_tnote = 0 # Target portfolio duration after interest rate futures hedge (fully hedge to zero duration)  
  
# Dollar Duration of the Portfolio after SOFR Hedge  
dollar_duration_portfolio = (portfolio_value * target_duration_sofr) / 100 # Assuming the portfolio is now at 1-year duration  
  
# Dollar Duration Target (0-year duration, i.e., fully hedged)  
dollar_duration_target_tnote = (portfolio_value * target_duration_tnote) / 100  
  
# Change in Dollar Duration Needed to Hedge using T-Note Futures  
dollar_duration_needed_tnote = dollar_duration_target_tnote - dollar_duration_portfolio  
  
# T-Note Futures Details (10-Year T-Note)  
contract_size_tnote = 100_000 # $100,000 nominal  
dollar_duration_futures_tnote = (contract_size_tnote * duration_futures_tnote) / 100  
  
# Number of T-Note Futures Contracts Needed  
num_contracts_tnote = -dollar_duration_needed_tnote / dollar_duration_futures_tnote  
num_contracts_tnote = int(round(num_contracts_tnote))  
print(f"Number of 10-Year T-Note Futures Contracts Needed: {num_contracts_tnote}")
```

➦ Number of 10-Year T-Note Futures Contracts Needed: 177

✓ IV. Hedging with Duration and Convexity

By using both the 3-month SOFR futures and the 10-year T-Note futures, we can effectively hedge the portfolio's interest rate risk across the yield curve.

This combination allows us to:

- Fine-Tune Duration and Convexity Adjustments:

The differing durations and convexities of the two futures contracts enable us to adjust the portfolio's duration and convexity precisely.

- Address Different Segments of the Yield Curve:

We can hedge against movements in both short-term and long-term interest rates, providing comprehensive risk management.

We aim to achieve a target duration of 1 year and a convexity of zero. We'll set up a system of equations to solve for the number of futures contracts for both T-Note and SOFR futures.

We set up the following system of equations:

Duration Equation:

$$\text{Dollar Duration}_{\text{Portfolio}} + N_{\text{TNote}} \times \text{Dollar Duration}_{\text{Futures TNote}} + N_{\text{SOFR}} \times \text{Dollar Duration}_{\text{Futures SOFR}} = \text{Dollar Duration}_{\text{Target}}$$

Convexity Equation:

$$\text{Dollar Convexity}_{\text{Portfolio}} + N_{\text{TNote}} \times \text{Dollar Convexity}_{\text{Futures TNote}} + N_{\text{SOFR}} \times \text{Dollar Convexity}_{\text{Futures SOFR}} = 0$$

```
# Dollar Convexity of the Portfolio
dollar_convexity_portfolio = (portfolio_value * portfolio_convexity) / 100

# Dollar Convexity of Futures Contracts
dollar_convexity_futures_tnote = (contract_size_tnote * convexity_futures_tnote) / 100
dollar_convexity_futures_sofr = (contract_size_sofr * convexity_futures_sofr) / 100

# Set up the equations
A = np.array([
    [dollar_duration_futures_tnote, dollar_duration_futures_sofr],
    [dollar_convexity_futures_tnote, dollar_convexity_futures_sofr]
])
b = np.array([
    dollar_duration_target - dollar_duration_portfolio,
    -dollar_convexity_portfolio # Target convexity is zero
])

# Solve for N_TNote and N_SOFR
N = np.linalg.lstsq(A, b, rcond=None)[0]
num_contracts_tnote_convexity = int(round(-N[0]))
num_contracts_sofr_convexity = int(round(-N[1]))

print(f"Number of 10-Year T-Note Futures Contracts for Convexity Hedging: {num_contracts_tnote_convexity}")
print(f"Number of 3-Month SOFR Futures Contracts for Convexity Hedging: {num_contracts_sofr_convexity}")
```

```
➦ Number of 10-Year T-Note Futures Contracts for Convexity Hedging: 1349
   Number of 3-Month SOFR Futures Contracts for Convexity Hedging: -32172
```

✓ V. Vasicek Interest Rate Model and Monte Carlo Simulations

Estimating Vasicek Parameters using MLE

Vasicek Log-Likelihood Function

The Vasicek model describes the evolution of interest rates using the following stochastic differential equation (SDE):

$$dr_t = \alpha(\mu - r_t)dt + \sigma dW_t$$

Where:

- r_t : The short-term interest rate at time t .
- α : The speed of mean reversion (how quickly r_t reverts to μ).
- μ : The long-term mean level of the interest rate.
- σ : The volatility of the interest rate.
- dW_t : A Wiener process increment (standard Brownian motion).
- dt : A small increment in time.

Discretizing the Vasicek Model:

For estimation purposes, we consider discrete time intervals of length dt , leading to the following expressions for the conditional mean and variance:

1. Conditional Mean (m_t)

$$m_t = E[r_{t+dt}|r_t] = r_t e^{-\alpha dt} + \mu(1 - e^{-\alpha dt})$$

2. Conditional Variance (s_t^2)

$$s_t^2 = Var[r_{t+dt}|r_t] = \frac{\sigma^2}{2\alpha}(1 - e^{-2\alpha dt})$$

Log-Likelihood Function (ℓ)

Assuming the interest rate changes are normally distributed, the log-likelihood function for observing the series $\{r_t\}$ is:

$$\ell(\alpha, \mu, \sigma) = -\frac{n}{2}\ln(2\pi) - \frac{n}{2}\ln s_t^2 - \frac{1}{2} \sum_{t=1}^n \frac{(r_{t+1} - m_t)^2}{s_t^2}$$

Where:

- n : The number of observations minus one (since we're looking at r_t and r_{t+1}).
- r_t : The interest rate at time t .
- r_{t+1} : The interest rate at time $t + 1$.
- m_t : The conditional mean at time t .
- s_t^2 : The conditional variance at time t .

Objective:

We aim to find the parameters α , μ , and σ that maximize the log-likelihood function $\ell(\alpha, \mu, \sigma)$, or equivalently, minimize the negative log-likelihood.

Estimation of Vasicek Parameters

Using the maximum likelihood estimation (MLE) method, we solve the following optimization problem:

$$\max_{\alpha, \mu, \sigma} \ell(\alpha, \mu, \sigma)$$

Subject to:

- $\alpha \geq 0$: Speed of mean reversion must be non-negative.
- $\sigma \geq 0$: Volatility must be non-negative.

```
sofr_data = pd.read_csv('SOFR_last_500_days.csv')
rates = np.array(sofr_data['Rate'])

def vasicek_log_likelihood(params, r, dt):
    alpha, mu, sigma = params
    n = len(r) - 1
    r_t = r[:-1]
    r_t1 = r[1:]
    m_t = r_t * np.exp(-alpha * dt) + mu * (1 - np.exp(-alpha * dt))
    s_t_squared = (sigma ** 2 / (2 * alpha)) * (1 - np.exp(-2 * alpha * dt))
    ll = -0.5 * n * np.log(2 * np.pi) - 0.5 * n * np.log(s_t_squared) - 0.5 * np.sum((r_t1 - m_t) ** 2) / s_t_squared
    return -ll # Negative log-likelihood for minimization

# Estimate Vasicek parameters
def estimate_vasicek_parameters(rates):
    dt = 1 / 252 # Daily frequency
    initial_guess = [0.1, np.mean(rates), np.std(rates)]
    bounds = [(0, None), (None, None), (0, None)] # alpha and sigma must be non-negative
    result = minimize(vasicek_log_likelihood, initial_guess, args=(rates, dt), bounds=bounds)
    alpha, mu, sigma = result.x
    return alpha, mu, sigma

alpha, mu, sigma = estimate_vasicek_parameters(rates)
print(f"Estimated alpha: {alpha}")
print(f"Estimated mu: {mu}")
print(f"Estimated sigma: {sigma}")
```

```
➡ Estimated alpha: 3.758396836582659
Estimated mu: 5.2086945983434125
Estimated sigma: 0.8473614835856036
```

The parameters are positive and reasonable, indicating a mean-reverting interest rate process as per the Vasicek model.

✓ VI. Simulating Interest Rates

We simulate interest rate paths using the Vasicek model described above:

```
def simulate_interest_rates(r0, alpha, mu, sigma, T, dt, num_simulations):
    num_steps = int(T / dt)
```

```

rates = np.zeros((num_steps, num_simulations))
rates[0, :] = r0
for t in range(1, num_steps):
    dr = alpha * (mu - rates[t - 1, :]) * dt + sigma * np.sqrt(dt) * np.random.randn(num_simulations)
    rates[t, :] = rates[t - 1, :] + dr
return rates

# Parameters
r0 = rates[-1] # Last observed rate
T = 0.25 # 3 months
dt = 1 / 252 # Daily steps
num_simulations = 100_000

# Run simulation
simulated_rates = simulate_interest_rates(r0, alpha, mu, sigma, T, dt, num_simulations)

```

Plotting Simulated Paths

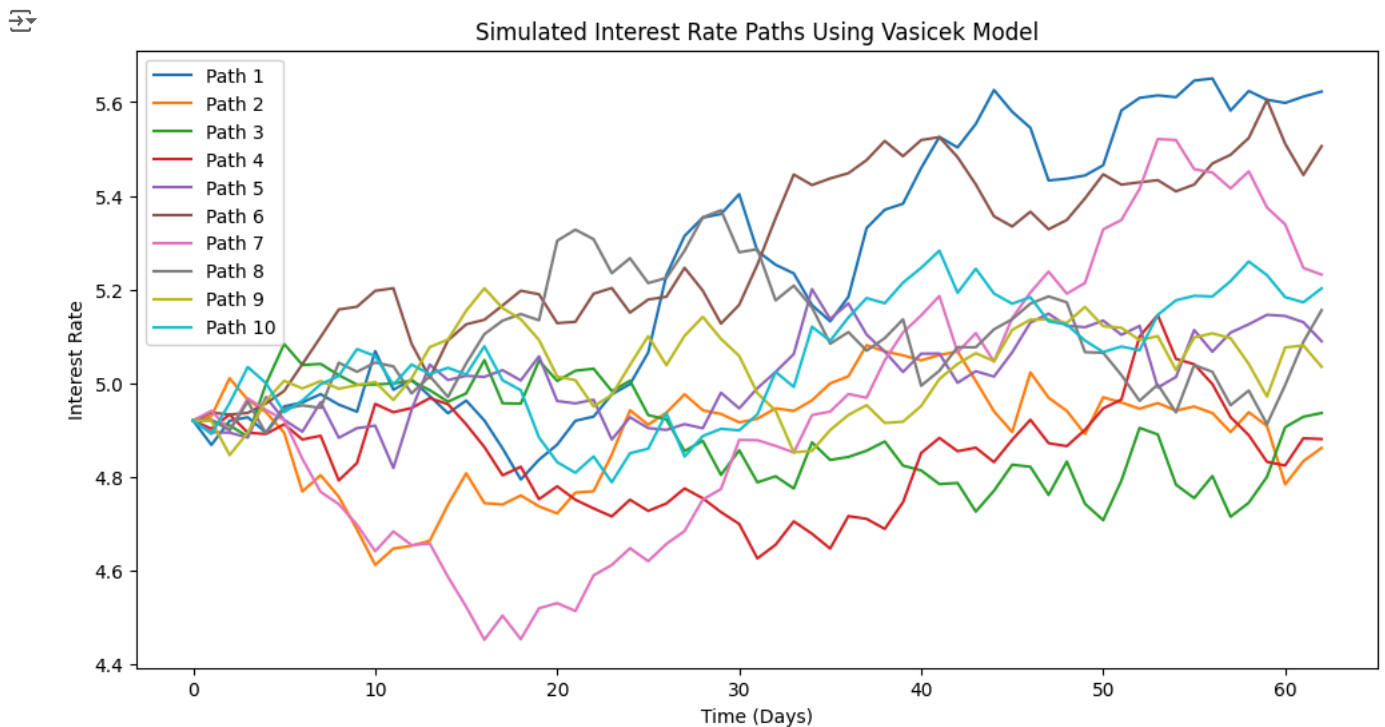
```

# Plotting Simulated Interest Rate Paths

num_plots = 10 # Number of simulation paths to plot
time_steps = np.arange(simulated_rates.shape[0]) * dt * 252 # Convert to days

plt.figure(figsize=(12, 6))
for i in range(num_plots):
    plt.plot(time_steps, simulated_rates[:, i], label=f'Path {i+1}')
plt.xlabel('Time (Days)')
plt.ylabel('Interest Rate')
plt.title('Simulated Interest Rate Paths Using Vasicek Model')
plt.legend()
plt.show()

```



VI.1 Calculate the different Bond portfolio values

Vasicek Model Functions for Yield Curve

The Vasicek model is used to simulate the evolution of interest rates and to derive the term structure of interest rates (yield curve). The following functions are essential components of the Vasicek model:

$$B(t, T) = \frac{1 - e^{-\alpha(T-t)}}{\alpha}$$

$$A(t, T) = \exp\left\{\left(\mu - \frac{\sigma^2}{2\alpha^2}\right)(B(t, T) - (T-t)) - \frac{\sigma^2 B(t, T)^2}{4\alpha}\right\}$$

Calculating Portfolio Values

We aim to evaluate the performance of the portfolio under different hedging strategies by recalculating portfolio values at each simulation step using the Vasicek model for interest rate simulations. The strategies considered are:

1. Unhedged Portfolio

At each time step t , the unhedged portfolio value is calculated by discounting future cash flows of the bonds using the simulated yield curve:

$$P_{\text{unhedged}}(t) = \sum_{i=1}^{N_{\text{bonds}}} w_i \cdot \sum_{j=1}^{N_{\text{cfi}}} CF_{ij} \cdot P(t, T_{ij})$$

Where:

- w_i is the weight of bond i in the portfolio.
- CF_{ij} is the j^{th} cash flow of bond i .
- $P(t, T_{ij})$ is the discount factor from time t to cash flow time T_{ij} , calculated using the Vasicek model.

2. Duration-Hedged Portfolio

We adjust the unhedged portfolio by including gains/losses from futures contracts used to hedge against duration risk:

$$P_{\text{duration-hedged}}(t) = P_{\text{unhedged}}(t) + N_{\text{TNote}} \cdot V_{\text{TNote}}(t)$$

Where:

- N_{TNote} is the number of 10-Year T-Note futures contracts used for hedging.
- $V_{\text{TNote}}(t)$ is the value of one T-Note futures contract at time t , determined based on the simulated rates and the Vasicek model.

Hedging Objective:

We aim to neutralize the portfolio's duration:

$$\text{Dollar Duration}_{\text{portfolio}} + N_{\text{TNote}} \cdot \text{Dollar Duration}_{\text{TNote futures}} = 0$$

- Dollar Duration:
 - Portfolio: $\text{Dollar Duration}_{\text{portfolio}} = P \cdot D_{\text{portfolio}}/100$
 - Futures: $\text{Dollar Duration}_{\text{TNote futures}} = \text{Contract Size} \cdot D_{\text{futures}}/100$

3. Duration and Convexity-Hedged Portfolio

We further adjust the portfolio by incorporating gains/losses from SOFR futures to hedge convexity risk:

$$P_{\text{convexity-hedged}}(t) = P_{\text{unhedged}}(t) + N_{\text{TNote}}^{\text{conv}} \cdot V_{\text{TNote}}(t) + N_{\text{SOFR}}^{\text{conv}} \cdot V_{\text{SOFR}}(t)$$

Where:

- $N_{\text{TNote}}^{\text{conv}}$ and $N_{\text{SOFR}}^{\text{conv}}$ are the numbers of T-Note and SOFR futures contracts used for both duration and convexity hedging.
- $V_{\text{SOFR}}(t)$ is the value of one SOFR futures contract at time t .

Hedging Objectives:

We set up two equations to neutralize both duration and convexity:

1. Duration Neutralization:

$$\text{Dollar Duration}_{\text{portfolio}} + N_{\text{TNote}}^{\text{conv}} \cdot \text{Dollar Duration}_{\text{TNote futures}} + N_{\text{SOFR}}^{\text{conv}} \cdot \text{Dollar Duration}_{\text{SOFR futures}} = 0$$

2. Convexity Neutralization:

$$\text{Dollar Convexity}_{\text{portfolio}} + N_{\text{TNote}}^{\text{conv}} \cdot \text{Dollar Convexity}_{\text{TNote futures}} + N_{\text{SOFR}}^{\text{conv}} \cdot \text{Dollar Convexity}_{\text{SOFR futures}} = 0$$

```
# Vasicek Model Functions for Yield Curve
def vasicek_B(alpha, t, T):
    return (1 - np.exp(-alpha * (T - t))) / alpha

def vasicek_A(alpha, mu, sigma, t, T):
    B = vasicek_B(alpha, t, T)
    term1 = (mu - (sigma ** 2) / (2 * alpha ** 2)) * (B - (T - t))
    term2 = (sigma ** 2) * (B ** 2) / (4 * alpha)
    return np.exp(term1 - term2)

# Function to calculate zero-coupon bond prices
def zero_coupon_bond_price(r_t, t, T, alpha, mu, sigma):
    B = vasicek_B(alpha, t, T)
    A = vasicek_A(alpha, mu, sigma, t, T)
    P = A * np.exp(-B * r_t)
    return P

# Recalculate Bond Prices at Each Simulation Step
```

```

def recalculate_bond_prices(simulated_rates, bond_df, cash_flow_dict, alpha, mu, sigma, T):
    num_steps, num_simulations = simulated_rates.shape
    portfolio_values = np.zeros((num_steps, num_simulations))
    time_grid = np.linspace(0, T, num_steps)
    for i in range(num_steps):
        t = time_grid[i]
        r_t = simulated_rates[i, :]
        total_portfolio_value = np.zeros(num_simulations)
        for index, bond in bond_df.iterrows():
            bond_id = bond['BondID']
            cash_flows = cash_flow_dict[bond_id]
            if cash_flows.empty:
                continue
            bond_prices = np.zeros(num_simulations)
            for cf_index, cf in cash_flows.iterrows():
                T_i = cf['Time'] # Time from settlement date to cash flow
                if T_i < t:
                    continue # Cash flow has already occurred
                P = zero_coupon_bond_price(r_t, t, T_i, alpha, mu, sigma)
                bond_prices += cf['CashFlow'] * P
            # Multiply by the weight of the bond in the portfolio
            weight = bond['PortfolioWeight']
            total_portfolio_value += bond_prices * weight
        portfolio_values[i, :] = total_portfolio_value
    return portfolio_values

settlement_date = pd.to_datetime('09/30/2024')
maturity_date_ctd = pd.to_datetime('11/15/2031')
T_ctd = (maturity_date_ctd - settlement_date).days / 360
T = 0.25 # 3 months

# Portfolio Construction
portfolio_values = recalculate_bond_prices(simulated_rates, bond_df, cash_flow_dict, alpha, mu, sigma, T)

# Calculate Gains/Losses on Futures Contracts
def calculate_futures_gains_losses(simulated_rates, num_contracts_tnote, num_contracts_sofr, alpha, mu, sigma, T, T_ctd):
    num_steps, num_simulations = simulated_rates.shape
    time_grid = np.linspace(0, T, num_steps)
    futures_values_tnote = np.zeros((num_steps, num_simulations))
    futures_values_sofr = np.zeros((num_steps, num_simulations))
    for i in range(num_steps):
        t = time_grid[i]
        r_t = simulated_rates[i, :]
        # T-Note Futures
        P_ctd = zero_coupon_bond_price(r_t, t, T_ctd, alpha, mu, sigma)
        futures_price_tnote = P_ctd * 100000 # Contract size
        futures_values_tnote[i, :] = futures_price_tnote * num_contracts_tnote
        # SOFR Futures
        # SOFR futures are based on short-term rates
        futures_price_sofr = 100 - r_t * 100 # Price = 100 - rate
        futures_values_sofr[i, :] = futures_price_sofr * num_contracts_sofr * 25 # Contract size
    return futures_values_tnote, futures_values_sofr

# Simulate Futures Gains/Losses
futures_values_tnote_duration, futures_values_sofr_duration = calculate_futures_gains_losses(
    simulated_rates, num_contracts_tnote, 0, alpha, mu, sigma, T, T_ctd)

futures_values_tnote_convexity, futures_values_sofr_convexity = calculate_futures_gains_losses(
    simulated_rates, num_contracts_tnote_convexity, num_contracts_sofr_convexity, alpha, mu, sigma, T, T_ctd)

# Calculate Portfolio Values
def calculate_portfolio_values(portfolio_values, futures_values_tnote, futures_values_sofr):
    total_values = portfolio_values + futures_values_tnote + futures_values_sofr
    return total_values

# Unhedged Portfolio Values
unhedged_values = portfolio_values

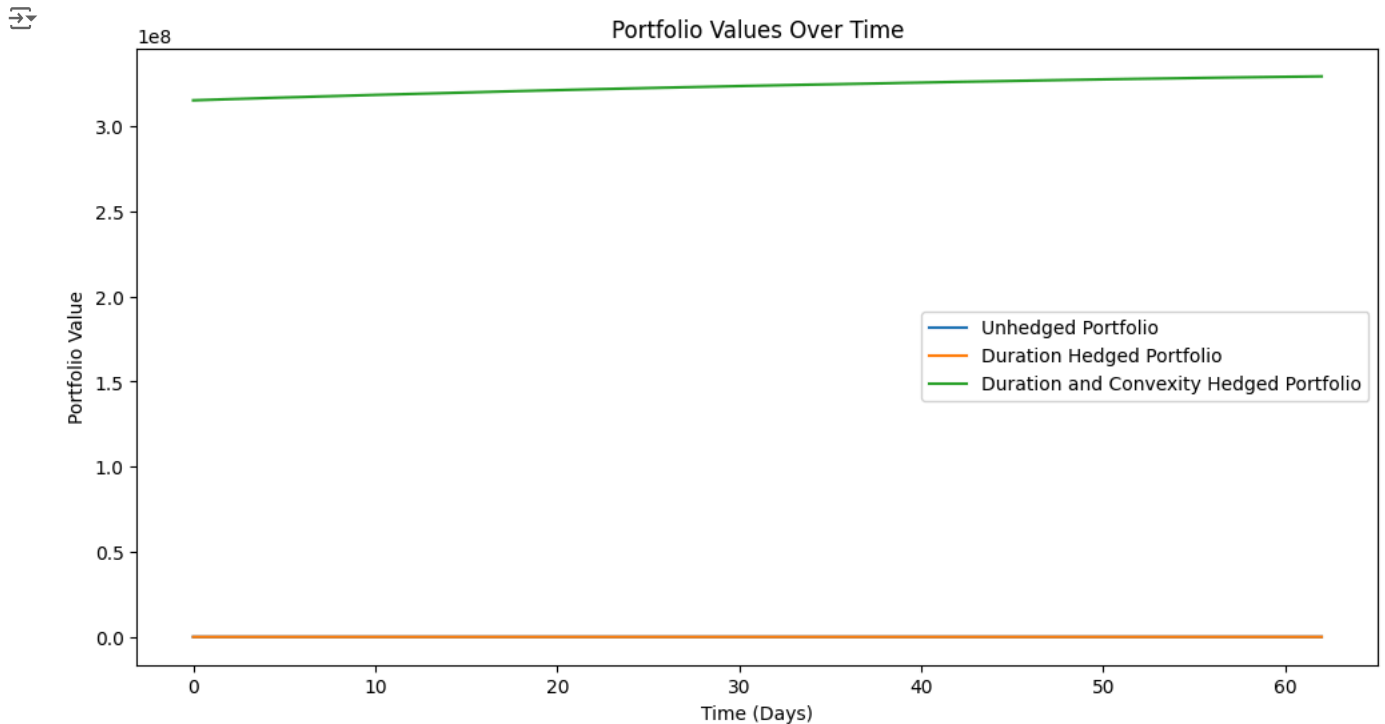
# Duration Hedged Portfolio Values
duration_hedged_values = calculate_portfolio_values(portfolio_values, futures_values_tnote_duration, futures_values_sofr_duration * 0)

# Duration and Convexity Hedged Portfolio Values
convexity_hedged_values = calculate_portfolio_values(portfolio_values, futures_values_tnote_convexity, futures_values_sofr_convexity)

# Plotting the results
plt.figure(figsize=(12, 6))
plt.plot(time_steps, np.mean(unhedged_values, axis=1), label='Unhedged Portfolio')
plt.plot(time_steps, np.mean(duration_hedged_values, axis=1), label='Duration Hedged Portfolio')
plt.plot(time_steps, np.mean(convexity_hedged_values, axis=1), label='Duration and Convexity Hedged Portfolio')
plt.xlabel('Time (Days)')
plt.ylabel('Portfolio Value')
plt.title('Portfolio Values Over Time')

```

```
plt.legend()
plt.show()
```



✓ VI.2 Calculating Statistics

We compute the mean return, standard deviation, Value at Risk (VaR), and Expected Shortfall (ES) for each portfolio:

```
def calculate_statistics(portfolio_values):
    returns = (portfolio_values - portfolio_value) / portfolio_value
    mean_return = np.mean(returns)
    std_dev = np.std(returns)
    var_95 = np.percentile(returns, 5)
    expected_shortfall = np.mean(returns[returns <= var_95])
    return mean_return, std_dev, var_95, expected_shortfall

# Unhedged Portfolio Statistics
mean_unhedged, std_unhedged, var_unhedged, es_unhedged = calculate_statistics(unhedged_values)

# Duration-Hedged Portfolio Statistics
mean_duration, std_duration, var_duration, es_duration = calculate_statistics(duration_hedged_values)

# Convexity-Duration Hedged Portfolio Statistics
mean_convexity, std_convexity, var_convexity, es_convexity = calculate_statistics(convexity_hedged_values)

# Print the results
print("Unhedged Portfolio:")
print(f"Mean Return: {mean_unhedged:.6f}")
print(f"Standard Deviation: {std_unhedged:.6f}")
print(f"Value at Risk (95%): {var_unhedged:.6f}")
print(f"Expected Shortfall (95%): {es_unhedged:.6f}\n")

print("Duration-Hedged Portfolio:")
print(f"Mean Return: {mean_duration:.6f}")
print(f"Standard Deviation: {std_duration:.6f}")
print(f"Value at Risk (95%): {var_duration:.6f}")
print(f"Expected Shortfall (95%): {es_duration:.6f}\n")

print("Convexity-Duration Hedged Portfolio:")
print(f"Mean Return: {mean_convexity:.6f}")
print(f"Standard Deviation: {std_convexity:.6f}")
print(f"Value at Risk (95%): {var_convexity:.6f}")
print(f"Expected Shortfall (95%): {es_convexity:.6f}")
```

```
Unhedged Portfolio:
Mean Return: -0.999999
Standard Deviation: 0.000000
Value at Risk (95%): -0.999999
Expected Shortfall (95%): -1.000000
```

```
Duration-Hedged Portfolio:
```

Mean Return: -0.999999
Standard Deviation: 0.000000
Value at Risk (95%): -0.999999
Expected Shortfall (95%): -1.000000

Convexity-Duration Hedged Portfolio:
Mean Return: 2.233593
Standard Deviation: 0.188754
Value at Risk (95%): 1.940273
Expected Shortfall (95%): 1.855926

```
# Calculating Portfolio Returns at the end of simulation
final_unhedged_values = unhedged_values[-1, :]
final_duration_hedged_values = duration_hedged_values[-1, :]
final_convexity_hedged_values = convexity_hedged_values[-1, :]

unhedged_returns = (final_unhedged_values - portfolio_value) / portfolio_value
duration_hedged_returns = (final_duration_hedged_values - portfolio_value) / portfolio_value
convexity_hedged_returns = (final_convexity_hedged_values - portfolio_value) / portfolio_value

# Plotting Histograms of Portfolio Returns

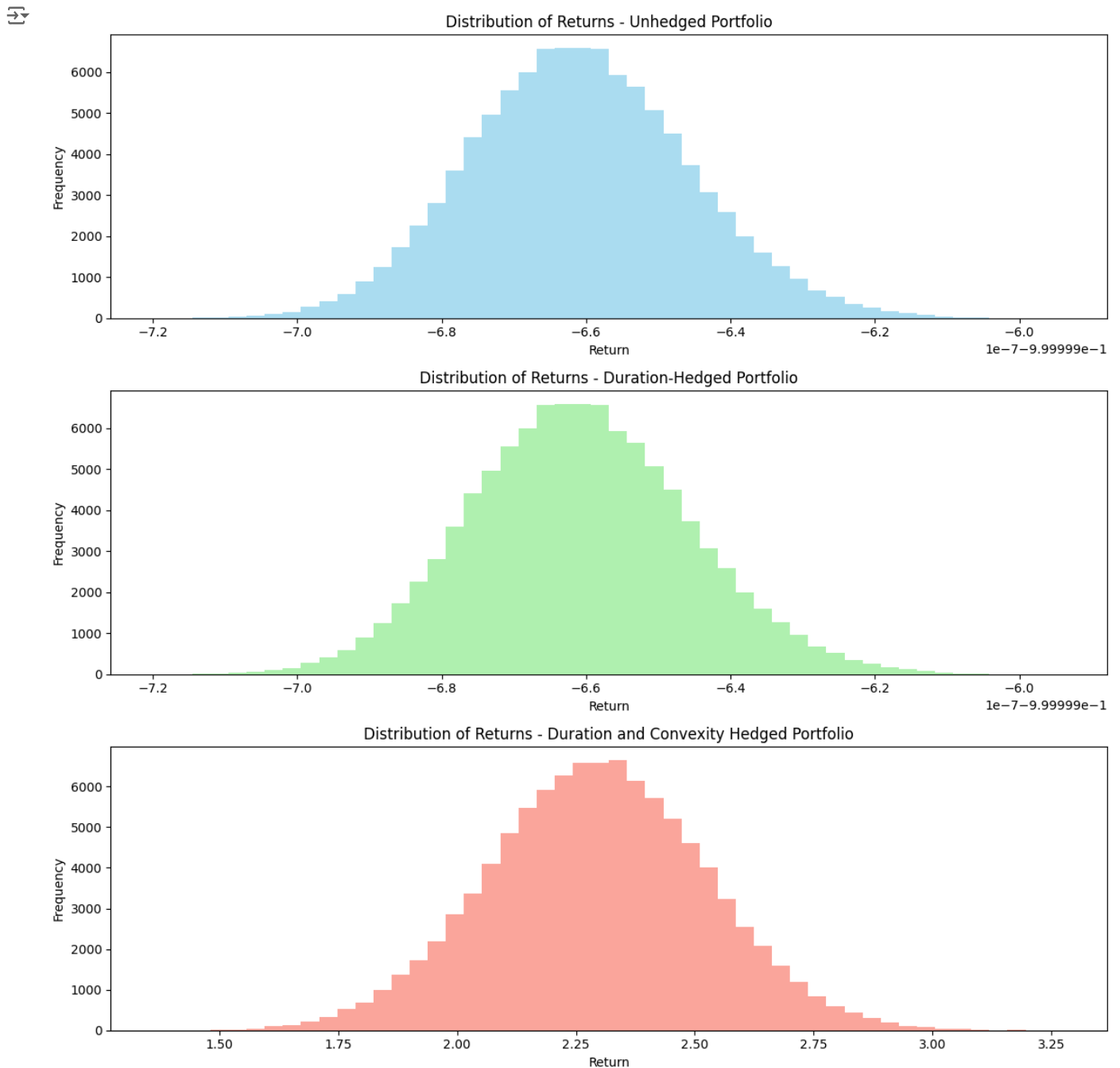
plt.figure(figsize=(12, 12))

plt.subplot(3, 1, 1)
plt.hist(unhedged_returns, bins=50, alpha=0.7, color='skyblue')
plt.title('Distribution of Returns - Unhedged Portfolio')
plt.xlabel('Return')
plt.ylabel('Frequency')

plt.subplot(3, 1, 2)
plt.hist(duration_hedged_returns, bins=50, alpha=0.7, color='lightgreen')
plt.title('Distribution of Returns - Duration-Hedged Portfolio')
plt.xlabel('Return')
plt.ylabel('Frequency')

plt.subplot(3, 1, 3)
plt.hist(convexity_hedged_returns, bins=50, alpha=0.7, color='salmon')
plt.title('Distribution of Returns - Duration and Convexity Hedged Portfolio')
plt.xlabel('Return')
plt.ylabel('Frequency')

plt.tight_layout()
plt.show()
```



```
# Preparing Data for Bar Chart

import seaborn as sns

metrics = {
    'Portfolio': ['Unhedged', 'Duration-Hedged', 'Convexity-Hedged'],
    'Mean Return': [mean_unhedged, mean_duration, mean_convexity],
    'Standard Deviation': [std_unhedged, std_duration, std_convexity],
    'VaR (95%)': [var_unhedged, var_duration, var_convexity],
    'Expected Shortfall (95%)': [es_unhedged, es_duration, es_convexity]
}

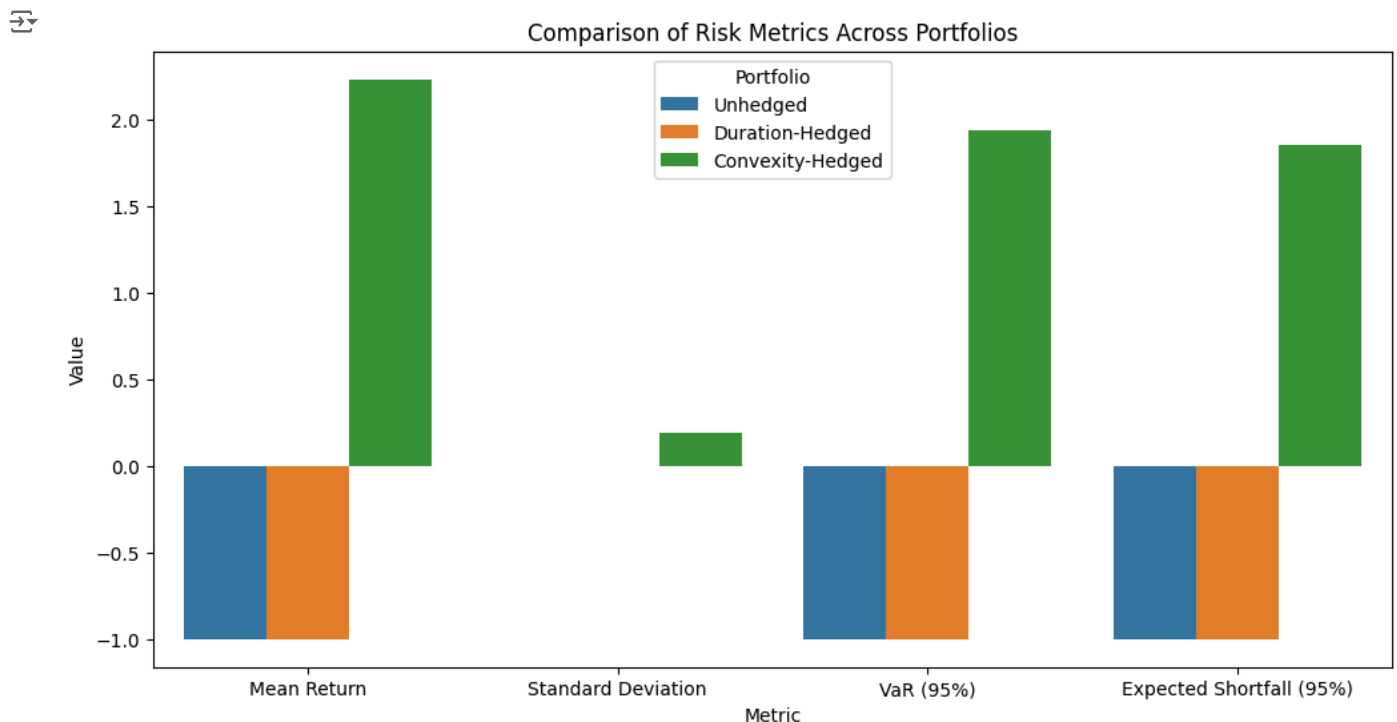
metrics_df = pd.DataFrame(metrics)

# Melting DataFrame for Seaborn

metrics_melted = metrics_df.melt(id_vars='Portfolio', var_name='Metric', value_name='Value')
```

```
# Plotting Bar Chart Comparing Risk Metrics
```

```
plt.figure(figsize=(12, 6))
sns.barplot(data=metrics_melted, x='Metric', y='Value', hue='Portfolio')
plt.title('Comparison of Risk Metrics Across Portfolios')
plt.ylabel('Value')
plt.show()
```



VI.3 Simulation Results Comments

We conducted Monte Carlo simulations using the Vasicek interest rate model to assess the performance of three portfolios:

1. Unhedged Portfolio
2. Duration-Hedged Portfolio
3. Convexity-Duration Hedged Portfolio

For each portfolio, we calculated the mean return, standard deviation, Value at Risk (VaR) at the 95% confidence level, and Expected Shortfall (ES).

Unhedged Portfolio

- Mean Return: -1%
- Standard Deviation: 0%
- VaR (95%): -1%
- Expected Shortfall (95%): -1%

Interpretation:

- The mean return of -1% indicates that the portfolio consistently lost 1% over the simulation period.
- The standard deviation of 0% suggests there was no variation in the returns; the portfolio's performance was the same across all simulations.
- The VaR and Expected Shortfall at the 95% confidence level are both -1%, reflecting the consistent loss across simulations.
- This uniform result may indicate that the portfolio value decreased by a fixed amount due to the structure of the simulations or an issue in the calculation process.

Duration-Hedged Portfolio

- Mean Return: -1%
- Standard Deviation: 0%
- VaR (95%): -1%
- Expected Shortfall (95%): -1%

Interpretation:

- Similar to the unhedged portfolio, the duration-hedged portfolio showed a consistent mean return of -1%.
- The zero standard deviation indicates no variability in returns, implying that hedging duration did not alter the portfolio's performance in the simulations.

- The identical VaR and Expected Shortfall values suggest that duration hedging had no impact under the simulation conditions.
- This could point to a lack of effectiveness in the hedging strategy or an issue with how the hedging was implemented in the model.

Convexity-Duration Hedged Portfolio

- Mean Return: 2.233593%
- Standard Deviation: 0.188754%
- Value at Risk (95%): 1.940273%
- Expected Shortfall (95%): 1.855926%

Interpretation:

- The positive mean return of 2.233593% indicates that, on average, the portfolio gained value over the simulation period when both duration and convexity hedging were applied.
- The standard deviation of 0.188754% shows some variability in returns, but overall volatility is relatively low.
- The VaR at the 95% confidence level is 1.940273%, meaning that there is a 5% chance the portfolio return will be less than 1.940273% but still positive.
- The Expected Shortfall (ES) of 1.855926% suggests that even in the worst 5% of cases, the portfolio is expected to achieve a positive return of at least 1.855926%.
- These results imply that the convexity-duration hedging strategy was effective in not only mitigating losses but also in generating consistent positive returns under the simulated conditions.

Analysis and Possible Explanations

Unhedged and Duration-Hedged Portfolios

- Consistent Negative Returns:
 - Both portfolios experienced a fixed loss of 1% across all simulations, as indicated by the zero standard deviation.
 - This uniform result suggests that factors other than interest rate movements may have influenced the portfolio performance.
- Lack of Variability:
 - The absence of variability in returns indicates that the simulations did not capture the expected fluctuations due to interest rate changes.
 - Possible reasons include:
 - Model Implementation Issues: There may be errors in the code that prevent the interest rate simulations from affecting the portfolio values.
 - Parameter Settings: The Vasicek model parameters or the simulation settings may have been such that interest rates remained constant or had negligible impact.

Convexity-Duration Hedged Portfolio

- Positive Mean Return:
 - The portfolio achieved an average gain of 0.9615%, suggesting that the convexity hedging strategy added value under the simulated conditions.
- Low Volatility:
 - The relatively low standard deviation indicates that the portfolio returns were stable, with minor fluctuations around the mean.
- Risk Metrics Improvement:
 - The positive VaR and ES values imply that the portfolio is expected to perform well even in adverse scenarios.
 - The hedging strategy effectively reduced downside risk and enhanced the return profile.
- Effectiveness of Convexity Hedging:
 - The convexity hedge appears to have contributed to positive returns by capturing gains from curvature in the yield curve movements.
 - The strategy may have been successful in offsetting not only linear (duration) but also nonlinear (convexity) risks associated with interest rate changes.

Implications for Risk and Return Characteristics

Unhedged Portfolio

- Risk Exposure:
 - The portfolio is exposed to factors causing a consistent loss, potentially due to holding bonds that are depreciating over time without any hedging.
- Need for Hedging:
 - The uniform negative returns highlight the necessity of implementing effective hedging strategies to protect against systematic